

Deep Learning & GenAI — Detailed Concepts and Mathematics

Deep Learning & GenAI — Detailed Concepts and Mathematics

How to use this document. This is your deep study document. Each topic is explained from first principles: motivation → intuition → math → derivation → worked example → pitfalls. Read sequentially. Use file 02 (cheatsheet) for in-exam lookup and file 03 (mock paper) to test yourself.

Coverage map. Lec-3 RNN/LSTM/GRU · Lec-4 Seq2Seq+Attention · Lec-5 Self-Attention+FFN · Lec-6 Transformer/BERT · Lec-7 Pre-training/Fine-tuning · Lec-8 Prompting · Lec-9 IFT+RLHF · Lec-10 RLHF/Alignment · Pre-mid CNN + Segmentation.

Table of contents

1. CNN building blocks (Dropout, BatchNorm, Attention)
 2. Image Segmentation
 3. Language modelling fundamentals (n-grams, smoothing, perplexity)
 4. Word embeddings (word2vec, GloVe)
 5. RNN, LSTM, GRU — full derivations
 6. Seq2Seq + Attention (Bahdanau, Luong) — with worked example
 7. The Transformer — every component derived
 8. Encoder vs Decoder vs Encoder-Decoder (BERT, GPT, T5, BART)
 9. Pre-training and Fine-tuning (MLM, CLM, denoising, PEFT, LoRA)
 10. Prompting and In-context Learning
 11. Instruction Tuning and Synthetic Data
 12. RL fundamentals for LLMs (states, actions, returns, V/Q/A)
 13. Reward modelling (Bradley–Terry) — full derivation
 14. Policy gradients (REINFORCE), Importance Sampling, PPO/PPO-CLIP
 15. RLHF objective with KL regularisation; Constitutional AI; DPO
 16. Retrieval-Augmented Generation (REALM, RAG-Token, RAG-Sequence, FiD)
 17. Agentic frameworks (ReAct, Reflexion, ReWOO, Self-Refine, HuggingGPT)
 18. Worked numerical problems and longer solved examples
 19. Appendix: confusion pairs, full model-paper answer key
-

1. CNN building blocks

1.1 Convolution — revisited deeply

Why convolution and not dense layers?

A $224 \times 224 \times 3$ RGB image has 150,528 input features. A dense first layer with 1000 hidden units needs 150 million weights — too many to train, and *every* weight is independent. Convolution introduces three priors that match how images work:

1. **Locality.** A small filter (say $3 \times 3 \times 3 = 27$ weights) only sees a tiny patch at a time. Pixels far apart rarely interact directly in shallow layers — most useful patterns (edges, textures, parts) are local.
2. **Translation equivariance.** The same filter is swept across the whole image: a cat's ear detector should respond identically whether the ear is in the top-left or bottom-right.
3. **Parameter sharing.** Because of equivariance, the filter has the same parameters everywhere — total params for a $3 \times 3 \times 3 \rightarrow 64$ conv is $(3 \cdot 3 \cdot 3 + 1) \cdot 64 = 1,792$, not 150 million.

The math

For an input $X \in \mathbb{R}^{H \times W \times C_{in}}$, a filter bank $K \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ with stride s and zero-padding p :

$$Y[i, j, o] = b[o] + \sum_{u=0..k-1} \sum_{v=0..k-1} \sum_{c=0..C_{in}-1} K[u, v, c, o] \cdot X[s \cdot i + u - p, s \cdot j + v - p, c]$$

Output spatial size:

$$H_{out} = \text{floor}((H + 2p - k) / s) + 1, \quad \text{same for } W_{out}.$$

Receptive field

After L stacked 3×3 stride-1 convs, a single output neuron “sees” a $(2L+1) \times (2L+1)$ patch of the input. Pooling, dilation and strides increase this faster. The **effective receptive field** (Luo et al., 2016) is Gaussian-shaped and smaller than the theoretical maximum — important when designing segmentation architectures (you need enough RF to see the whole object).

Pooling

- **Max-pool** (2×2 stride 2): keeps strongest activation per window; introduces a small translation invariance. Halves spatial dims, doubles abstraction.
- **Average-pool / Global Average Pool (GAP):** GAP averages across $H \times W$; it replaces flatten+dense and is used in modern classifiers (ResNet) as a strong regulariser.

1.2 Dropout — full picture

The problem dropout solves

A neural net with millions of parameters can *memorise* the training set even after data augmentation. Worse, individual neurons can **co-adapt**: neuron A learns to be useful only because neuron B always fires with it. If B fails, A is useless. This brittle teamwork hurts generalisation.

The trick

At training time, for each forward pass, draw a fresh random mask $m_i \sim \text{Bernoulli}(1 - p)$ for every activation a_i and replace $a_i \leftarrow (m_i / (1 - p)) \cdot a_i$. The division by $(1 - p)$ (called **inverted dropout**) keeps the expected output unchanged, so we don't need to rescale at inference.

At test time, no dropout is applied — just plain forward pass.

Why it works (three complementary views)

1. **Ensemble of thinned networks.** Each forward pass uses a random sub-network. Over many training updates, you implicitly average $\sim 2^N$ such networks (where N is the number of dropped units), which is a Bayesian model averaging.
2. **No co-adaptation.** Any neuron's downstream consumer cannot rely on a specific peer being alive, so the consumer must learn *distributed* representations that are robust to absent features.
3. **Adaptive noise injection.** Dropout is approximately equivalent to multiplicative Gaussian noise; noise during training is a well-known regulariser (denoising autoencoders, weight noise, etc.).

Dropout in CNNs

Plain dropout on conv feature maps is *weak* because adjacent activations are highly correlated — the network just infers a dropped pixel from its neighbour. Better variants: - **SpatialDropout (Tompson et al.)**: drop entire feature-map *channels*. If the "ear detector" channel is dropped, no spatial neighbour can rescue it. - **DropBlock (Ghiasi et al.)**: drop contiguous spatial *blocks* (e.g. 7×7 patches). Removes semantically meaningful regions.

Practical placement

The canonical order in modern CNNs is `Conv → BN → ReLU → Dropout`. Placing dropout *before* BN can corrupt the batch statistics that BN estimates.

1.3 Batch Normalisation — derivation, intuition, and why it works

The problem BN solves

In a deep network, the distribution of inputs to layer L shifts as the parameters of layers $1..L-1$ are updated — Ioffe & Szegedy called this **internal covariate shift (ICS)**. The shift forces later layers to constantly re-adapt to changing input distributions, slowing training. ICS is also the reason aggressive learning rates can diverge: a single big update upstream cascades.

The math

For a mini-batch $B = \{x_1, \dots, x_m\}$ (one number per neuron per example):

$$\begin{aligned}\mu_B &= (1/m) \sum_i x_i \\ \sigma^2_B &= (1/m) \sum_i (x_i - \mu_B)^2 \\ \hat{x}_i &= (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)} && \leftarrow \text{standardise} \\ y_i &= \gamma \hat{x}_i + \beta && \leftarrow \text{learnable scale/shift}\end{aligned}$$

Per-channel for CNNs: μ and σ^2 are computed across (N, H, W) for each channel C , so γ and β are R^C .

Why it actually works (modern view)

The “ICS reduction” story is debated. Santurkar et al. (2018) showed BN’s real benefit is to **smooth the loss landscape**: it makes the loss locally Lipschitz, which means gradient descent can take much bigger steps without overshooting. Concretely, the gradient norm is bounded and changes slowly with the parameters.

Benefits in practice

1. **Higher learning rates.** Empirically you can use 5–10× larger LR’s.
2. **Faster convergence.** Often 3–10× fewer epochs to reach a given accuracy.
3. **Mild regularisation.** Each minibatch produces noisy μ and σ^2 , which inject stochasticity into activations.
4. **Less sensitivity to initialisation.** The standardisation removes scale dependence on the previous layer’s weight init.

Inference

You can’t standardise with batch statistics at inference (the user gives one input at a time). During training, you accumulate exponential moving averages of μ_B and σ^2_B :

$$\begin{aligned}\mu_{\text{running}} &\leftarrow \alpha \cdot \mu_{\text{running}} + (1 - \alpha) \cdot \mu_B \\ \sigma^2_{\text{running}} &\leftarrow \alpha \cdot \sigma^2_{\text{running}} + (1 - \alpha) \cdot \sigma^2_B\end{aligned}$$

At inference, use these running estimates. γ , β remain unchanged.

BN variants (know these by name)

Variant	Normalises over	Use case
BatchNorm	(N, H, W) per channel	CNNs with large batches
LayerNorm	(C) per (N, H, W) location	Transformers, RNNs
GroupNorm	$(G \text{ groups of channels}, H, W)$	small batches in CNNs
InstanceNorm	(H, W) per (N, C)	style transfer
RMSNorm	LN without mean centring	LLaMA-style LLMs (cheaper)

1.4 Attention in CNNs (Spatial / Channel / SE / CBAM)

Channel attention — the SE block (Hu et al., 2017)

Squeeze-and-Excitation asks: of the C feature channels coming out of a conv block, which ones are *globally* most relevant to the current input? It computes a scalar per channel:

1. **Squeeze** (global avg pool): $z_c = (1/HW) \sum_{i,j} F[i, j, c] \rightarrow z \in \mathbb{R}^C$.
2. **Excite** (bottleneck FC): $s = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot z))$, where $W_1 \in \mathbb{R}^{(C/r) \times C}$, $W_2 \in \mathbb{R}^{C \times (C/r)}$, reduction ratio r (default 16).
3. **Scale**: $F'[i, j, c] = s_c \cdot F[i, j, c]$.

Total extra params per SE block: $2 \cdot C^2 / r$. For $C=256$, $r=16$ that's 8192 — tiny.

Spatial attention

Asks: which (i, j) positions matter? Compute a 2-D map $M \in [0, 1]^{H \times W}$ from the feature map (e.g. via $\text{mean across } C \rightarrow 1 \times 1 \text{ conv} \rightarrow \text{sigmoid}$), then $F' = M \odot F$.

CBAM = Channel + Spatial in series

Apply channel attention first, then spatial attention. CBAM (Woo et al., 2018) consistently outperforms SE on classification and detection.

Spectral attention (hyperspectral imagery)

Same as channel attention conceptually, but the “channels” are spectral bands (NIR, Red, etc.). Lets the model up-weight bands that are most discriminative for the task — vegetation indices use NIR heavily, while urban segmentation may rely on visible bands.

2. Image Segmentation

2.1 The task hierarchy

- **Semantic segmentation.** Per-pixel class label. “Every pixel of every cat is class `cat`.”
- **Instance segmentation.** Per-pixel class + instance ID. “Cat#1 vs Cat#2 are separated.”
- **Panoptic segmentation.** Unified: `stuff` (sky, road — uncountable, no instances) + `things` (cars, people — countable, instances).

2.2 Classic architectures

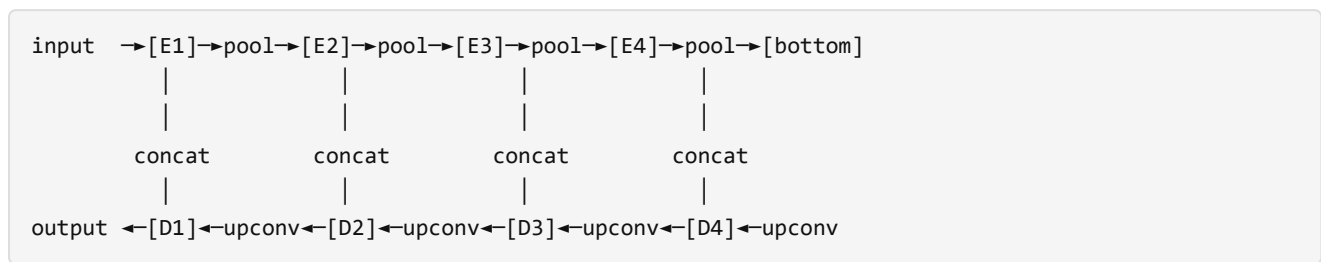
FCN (Fully Convolutional Network, Long et al., 2015)

Take a classifier (VGG, ResNet), strip the FC layers, replace with 1×1 convs, then **upsample** the coarse output back to input resolution using **transposed convolutions**. Adds **skip connections** from early layers (high resolution, low semantics) to late layers (low resolution, high semantics) so the output has both fine boundaries and correct labels.

U-Net (Ronneberger et al., 2015)

Designed for biomedical images with tiny training sets. The architecture is symmetric: - **Encoder (contracting path)**: repeated $[\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}]$ blocks. Spatial dims halve, channels double. - **Decoder (expanding path)**: repeated $[\text{Up-conv} \rightarrow \text{Concat skip} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow$

Conv → ReLU]. The concat skip brings high-resolution encoder features to each decoder level so that boundary localisation is preserved.



DeepLab (Chen et al., v1/v2/v3)

Two key ideas: 1. **Atrous (dilated) convolution**. A 3×3 conv with dilation rate r has gaps of $r-1$ between filter taps; the receptive field grows without losing resolution and without adding parameters. 2. **ASPP (Atrous Spatial Pyramid Pooling)**. Parallel atrous convs with rates $\{6, 12, 18, 24\}$ aggregate context at multiple scales.

Mask R-CNN (He et al., 2017)

Faster R-CNN (object detection) + a parallel mask head. The crucial fix is **RoIAlign** instead of **RoIPool**: RoIPool snaps the region-of-interest to integer pixel coordinates twice (once for the region, once for each cell), introducing ~ 0.5 -pixel misalignments per snap. That's fine for boxes but ruinous for pixel-accurate masks. RoIAlign uses bilinear interpolation at non-integer coordinates, eliminating quantisation error.

2.3 Losses for segmentation

- **Pixel-wise cross-entropy**. Standard. Works when classes are balanced.
- **Weighted CE**. Multiply each pixel's CE by a class weight (often $\text{median_freq} / \text{class_freq}$).
- **Dice loss**. $L_{\text{Dice}} = 1 - 2|A \cap B| / (|A| + |B|)$. Directly optimises overlap; great for tiny foreground classes (lesions, defects).
- **Focal loss**. $-\alpha(1-p_t)^\gamma \log p_t$. Down-weights easy examples; useful in extreme imbalance (foreground $\ll$ background).
- **Tversky loss**. Generalises Dice with separate penalties for false positives and false negatives — tune for precision/recall trade-off.

2.4 Metrics

- **Pixel accuracy**. Fraction of correctly classified pixels. Misleading on imbalanced data: predicting all-background can give 95%+.
- **mIoU (mean IoU)**. For each class c , $\text{IoU}_c = \text{TP}_c / (\text{TP}_c + \text{FP}_c + \text{FN}_c)$. Average over classes. Standard headline metric (Pascal VOC, Cityscapes).
- **Dice / F1**. Same idea, different formula: $2 \text{TP} / (2 \text{TP} + \text{FP} + \text{FN})$. Often reported alongside IoU.

3. Language modelling fundamentals

3.1 Definition

A *language model* assigns a probability $P(w_1, w_2, \dots, w_T)$ to a sequence of tokens. By the chain rule of probability:

$$\begin{aligned} P(w_1, w_2, \dots, w_T) &= \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}) \\ &= \prod_{t=1}^T P(w_t \mid w_{\{<t\}}). \end{aligned}$$

Different LMs differ in *how* they model $P(w_t \mid w_{\{<t\}})$: - **n-gram LMs** truncate the history to the last $n-1$ tokens. - **RNN / LSTM LMs** summarise the history into a hidden state. - **Transformer LMs** attend to all previous tokens.

3.2 n-gram language models

Definition and MLE

A bigram ($n=2$) LM uses only the immediate predecessor:

$$P(w_t \mid w_{\{<t\}}) \approx P(w_t \mid w_{\{t-1\}}).$$

Maximum-likelihood estimate:

$$\hat{P}(w_t \mid w_{\{t-1\}}) = \text{count}(w_{\{t-1\}}, w_t) / \text{count}(w_{\{t-1\}}).$$

Worked example. From a tiny corpus “the cat sat. the dog ran. the cat ran.”: - $\text{count}(\text{the}, \text{cat}) = 2$, $\text{count}(\text{the}) = 3 \rightarrow \hat{P}(\text{cat} \mid \text{the}) = 2/3$. - $\text{count}(\text{the}, \text{dog}) = 1 \rightarrow \hat{P}(\text{dog} \mid \text{the}) = 1/3$.

Why n-grams break

1. **Sparsity.** Most n-grams in any test set are *never* seen in training, giving $P = 0$ — and one zero in a product zeros the whole sentence probability.
2. **No generalisation.** “the red car drove” and “the blue car drove” are independent events to an n-gram LM; no notion that “red” and “blue” are both colours.
3. **Fixed history.** A trigram cannot recall a subject mentioned 50 tokens ago.

These limits motivated neural LMs (Bengio 2003), then RNN/LSTM LMs, then Transformers.

3.3 Smoothing — full picture

Why smoothing matters

Even with a 10-billion-word corpus, most rare events are unobserved. If $\text{count}(w_{\{t-1\}}, w_t) = 0$, MLE gives zero — but the event might be plausible. Smoothing reassigns probability mass from seen events to unseen ones.

Add-k (Laplace) smoothing

$$\hat{P}(w_t \mid w_{\{t-1\}}) = (\text{count}(w_{\{t-1\}}, w_t) + k) / (\text{count}(w_{\{t-1\}}) + k \cdot V).$$

- $k = 1$: original Laplace.
- Simple, but biases probability away from common events too aggressively.

- For $V = 50,000$ and a rare context, the smoothed estimate of any unseen bigram is $1 / (\text{count} + V) \approx 2 \times 10^{-5}$ — same for *every* unseen continuation, which is wrong (some are clearly more plausible).

Good-Turing

The number of n -grams seen r times is N_r . Good-Turing re-estimates expected count of an item seen r times as:

$$r^* = (r + 1) \cdot N_{\{r+1\}} / N_r.$$

This shifts probability mass from frequent events to rare ones in a principled way (it's the MLE under a Dirichlet prior with hyperparameters tied to N_r).

Interpolation (Jelinek–Mercer)

Linearly mix higher- and lower-order models:

$$\hat{P}(w_t \mid w_{\{t-2\}}, w_{\{t-1\}}) = \lambda_3 \cdot P(w_t \mid w_{\{t-2\}}, w_{\{t-1\}}) + \lambda_2 \cdot P(w_t \mid w_{\{t-1\}}) + \lambda_1 \cdot P(w_t).$$

Tune λ s on a held-out set with EM. Robust baseline.

Back-off (Katz)

Use the highest-order n -gram if seen; otherwise discount and “back off” to the next-lower order. Conceptually cleaner than interpolation but needs careful normalisation.

Kneser–Ney — the SOTA classical method

KN combines two insights: 1. **Absolute discounting**. Subtract a fixed d from every nonzero count before normalising; distribute the discounted mass to lower order. 2. **Continuation probability**. When backing off, don't use the *raw* unigram count — instead use the *number of distinct contexts* in which the word appears. This fixes the **“Francisco” problem**: $\text{count}(\text{Francisco})$ is high because “San Francisco” is common, but *Francisco* is *not* a likely word in arbitrary contexts. KN replaces $P(\text{Francisco})$ with the count of *distinct bigram types* $(?, \text{Francisco})$, which is small.

Modified KN (Chen & Goodman) uses three discount values for counts of 1, 2, and ≥ 3 . It is the strongest classical smoothing.

3.4 Perplexity — what it really measures

Definition

For a held-out token sequence $w_1, \dots, w_N$:

$$\begin{aligned} \text{PPL}(W) &= P(w_1, \dots, w_N)^{-1/N} \\ &= \exp\left(-\frac{1}{N} \sum_t \log P(w_t \mid w_{\{<t\}})\right). \end{aligned}$$

Three equivalent interpretations

1. **Inverse geometric-mean probability.** If the model assigns probabilities p_1, p_2, p_3 to three tokens, $\text{PPL} = (p_1 \cdot p_2 \cdot p_3)^{-1/3}$.
2. **Cross-entropy in tokens.** Let $H = - (1/N) \sum \log P(w_t | w_{<t})$ (in nats). Then $\text{PPL} = e^H$. In bits, $\text{PPL} = 2^H$.
3. **Effective branching factor.** If $\text{PPL} = 100$, the model is as confused as a uniform distribution over 100 tokens. Smaller is better.

Worked numerical example

A model assigns probabilities 0.5, 0.25, 0.125 to a 3-token test sequence:

```
log probs = -0.693, -1.386, -2.079    (nats)
sum = -4.158
average = -1.386
H = 1.386 nats
PPL = e^{1.386} ≈ 4.0.
```

So the model is “as confused as a uniform 4-way choice” — sensible since the geometric mean of {0.5, 0.25, 0.125} is $1/4$.

Pitfalls when comparing perplexities

- Vocabularies must be the same. A model with 50k tokens cannot be perplexity-compared to one with 32k tokens without normalisation.
- Tokenisation must be the same. BPE vs SentencePiece vs word-level give wildly different PPLs.
- PPL on training set is useless — only held-out PPL counts.

4. Word embeddings

4.1 The problem

One-hot word vectors are V -dimensional ($V \approx 100k$) and orthogonal — every word is equally distant from every other. The vector for “king” carries no information about “queen”. We want **dense, low-dimensional, similarity-preserving** vectors.

4.2 word2vec — Skip-gram with Negative Sampling (SGNS)

Setup

Two embedding matrices: $V \in \mathbb{R}^{\{|V| \times d\}}$ for *focus* words, $U \in \mathbb{R}^{\{|V| \times d\}}$ for *context* words. The focus vector for word c is v_c ; context vector for word o is u_o .

Objective

For a focus word c and a context word o within a window of size k , we want $v_c \cdot u_o$ large. For random “noise” words n sampled from a unigram distribution, we want $v_c \cdot u_n$ small. Hence the **negative-sampling loss**:

$$\begin{aligned} L(c, o) &= -\log \sigma(u_o \cdot v_c) - \sum_{n=1}^K \log \sigma(-u_n \cdot v_c) \\ &= -\log \sigma(u_o \cdot v_c) - \sum_{n=1}^K \log(1 - \sigma(u_n \cdot v_c)). \end{aligned}$$

This is the binary classification “is (c, o) a true co-occurrence or noise?” Typical $K = 5-20$.

Negative-sampling distribution

Words are sampled with probability $P_{\text{neg}}(w) \propto \text{freq}(w)^{3/4}$. The $3/4$ power smooths the distribution: very frequent words (“the”, “of”) are still sampled often, but rare words get more relative chances than under raw frequency.

Sub-sampling of frequent words

With probability $1 - \sqrt[t]{t / \text{freq}(w)}$ (typically $t = 10^{-5}$), a token is discarded from the corpus before training. This stops “the” from dominating updates and accelerates training.

Why it learns useful structure

The dot product $u_o \cdot v_c$ is essentially the log-odds of (c, o) being a true co-occurrence vs noise. Levy & Goldberg (2014) proved that skip-gram with negative sampling implicitly factorises a shifted positive-PMI matrix:

$$v_c \cdot u_o \approx \log[P(c, o) / (P(c) \cdot P(o))] - \log K.$$

So word2vec learns PMI in low rank — exactly what we want from a count-based factorisation.

CBOW (the other word2vec)

Predict the **focus** word from the **mean of context word vectors**. Mirror image of skip-gram; usually slightly weaker on rare words because they get averaged out.

4.3 GloVe — global co-occurrence factorisation

Motivation

Skip-gram looks at one window at a time. GloVe instead looks at the entire co-occurrence matrix X where X_{ij} counts how often word j appears in the context of word i across the whole corpus.

Objective

$$J = \sum_{i, j : X_{ij} > 0} f(X_{ij}) \cdot (w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2.$$

- w_i is the focus vector, $\tilde{w}_j$ is the context vector.
- The fit term says: the dot product should equal $\log X_{ij}$ minus bias terms.
- $f(X_{ij}) = \min(1, (X_{ij}/x_{\text{max}})^\alpha)$ (typically $x_{\text{max}} = 100$, $\alpha = 3/4$) damps the contribution of very frequent pairs that would otherwise dominate the loss.

Why does the dot product approximate $\log X_{ij}$?

The ratio $P(k|i)/P(k|j)$ is large when k is more related to i than to j . Taking logs and constraining the model to be a function of $w_i - w_j$, one ends up requiring $w_i \cdot \tilde{w}_k - w_j \cdot \tilde{w}_k \approx \log P(k|i) - \log P(k|j)$, which is solved by $w_i \cdot \tilde{w}_k \approx \log P(k|i) + \text{const}$. Adding bias terms and back-substituting gives the GloVe objective. (Full derivation in Pennington et al., 2014, Sec. 3.)

Exam fact (model paper Q1.1)

GloVe uses **no negative sampling**. It directly minimises the weighted squared error to $\log X_{ij}$. People often confuse this because both word2vec and GloVe are similar in spirit; remember GloVe = global counts + weighted least squares.

4.4 Comparison and limitations

	word2vec	GloVe
Training signal	Local windows + negative samples	Global co-occurrence counts
Loss	NEG / NCE (binary classification)	Weighted least squares on log-counts
Global stats	Captured implicitly	Captured directly
Speed	Fast, online	Faster on huge corpora (matrix is precomputed)
Typical quality	Comparable	Comparable; depends on corpus

Both produce **static** embeddings (one vector per word, regardless of context). They cannot disambiguate “bank” (river vs financial). This limitation drove ELMo (Peters et al., 2018) and then BERT, where embeddings are *contextual*.

5. Recurrent networks: RNN, LSTM, GRU

5.1 Vanilla RNN

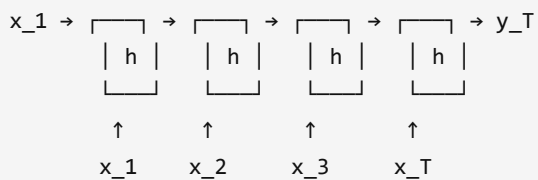
Equations

At each step t :

$$\begin{aligned}
 h_t &= \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h) && \text{(hidden update)} \\
 y_t &= W_{hy} \cdot h_t + b_y && \text{(output)} \\
 \hat{y}_t &= \text{softmax}(y_t) && \text{(for classification / LM)}
 \end{aligned}$$

- $x_t \in \mathbb{R}^m$ is the input token embedding.
- $h_t \in \mathbb{R}^n$ is the hidden state, carrying summary of $x_1, \dots, x_t$.
- Same weights W_{xh}, W_{hh}, W_{hy} at every step (recurrence $\leftrightarrow$ parameter sharing in time).

Diagram (unrolled)



Backprop Through Time (BPTT) — derivation

Suppose loss at the final step $L = L(y_T)$. We need $\partial L / \partial W_{hh}$. By chain rule:

$$\partial L / \partial W_{hh} = \sum_{t=1}^T (\partial L / \partial h_T) \cdot (\partial h_T / \partial h_t) \cdot (\partial h_t / \partial W_{hh}).$$

The middle factor expands as a product:

$$\begin{aligned} \partial h_T / \partial h_t &= \prod_{i=t+1}^T \partial h_i / \partial h_{i-1} \\ &= \prod_{i=t+1}^T W_{hh}^T \cdot \text{diag}(\tanh'(W_{xh} x_i + W_{hh} h_{i-1} + b_h)). \end{aligned}$$

Vanishing / exploding gradients

The product of $T - t$ matrices has spectral radius ρ^{T-t} where $\rho = \|W_{hh}^T \cdot \text{diag}(\tanh'(\cdot))\|$. - If $\rho < 1$, the gradient **vanishes** exponentially with the gap. The model cannot learn long-range dependencies. - If $\rho > 1$, the gradient **explodes** exponentially. - $\tanh'(z) \leq 1$ everywhere, so the diag matrix only contracts — *vanishing* is the dominant problem in practice.

Partial fixes for vanilla RNN: - **Gradient clipping** ($g \leftarrow g \cdot \min(1, \theta / \|g\|)$) — solves exploding only. - **Orthogonal init** of W_{hh} (preserves singular values). - **ReLU + identity init** (Le et al., 2015) — IRNN, partly works. - **None** truly fixes vanishing — that's why we have LSTM/GRU.

5.2 LSTM (Hochreiter & Schmidhuber, 1997)

Core idea

Add a **cell state** c_t that flows through time with very few changes; gates decide what to add, what to forget, and what to read out.

Equations (using $[h_{t-1}; x_t]$ to mean concatenation)

$f_t = \sigma(W_f \cdot [h_{t-1}; x_t] + b_f)$	(forget gate, $\in (0,1)^n$)
$i_t = \sigma(W_i \cdot [h_{t-1}; x_t] + b_i)$	(input gate)
$\hat{c}_t = \tanh(W_c \cdot [h_{t-1}; x_t] + b_c)$	(candidate cell update, $\in (-1,1)^n$)
$c_t = f_t \odot c_{t-1} + i_t \odot \hat{c}_t$	(new cell state)
$o_t = \sigma(W_o \cdot [h_{t-1}; x_t] + b_o)$	(output gate)
$h_t = o_t \odot \tanh(c_t)$	(new hidden state, fed to output and next step)

Why this fixes vanishing

Look at $\partial c_t / \partial c_{t-1}$:

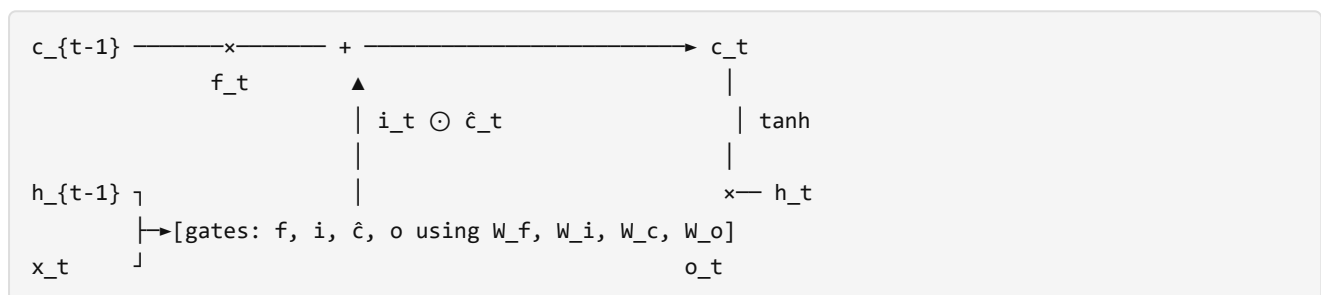
$$\partial c_t / \partial c_{t-1} = \text{diag}(f_t).$$

When $f_t \approx 1$, this is the identity — gradient flows backwards undamped. The cell state is a **constant-error carousel** when the forget gate is open. Compare to vanilla RNN where the corresponding derivative involves a $\tanh'(\cdot) \leq 1$ factor every step.

Each gate explained

- **Forget gate** f_t . “What proportion of each cell-state component should we keep?” 1 means keep, 0 means erase.
- **Input gate** i_t . “What proportion of the candidate update should we add?”
- **Candidate** $\hat{c}_t$. The *content* to add — bounded in $(-1, 1)$.
- **Output gate** o_t . “What proportion of the current cell state should we expose as the hidden state output?”

Diagram



Parameter count

For input dim m , hidden n : each of 4 gates has weight matrix of size $(m + n) \times n$ and bias of size n . Total: $4 \cdot ((m + n) \cdot n + n) = 4n(m + n + 1)$.

For $m = n = 512$: $4 \cdot 512 \cdot (512 + 512 + 1) = 4 \cdot 512 \cdot 1025 \approx 2.1 \text{ M}$ per LSTM layer.

Variants worth knowing

- **Peephole connections** (Gers & Schmidhuber, 2000): let the gates also see the cell state c_{t-1} directly.
- **Coupled forget/input** (Greff et al.): set $i_t = 1 - f_t$. Reduces params by 25% with marginal cost.
- **Bidirectional LSTM** (Graves & Schmidhuber): two LSTMs, one forward, one backward; their hidden states concatenated. Used for classification / NER, not generation.

5.3 GRU (Cho et al., 2014)

Equations

$$\begin{aligned}
 r_t &= \sigma(W_r \cdot [h_{t-1}; x_t]) && \text{(reset gate)} \\
 z_t &= \sigma(W_z \cdot [h_{t-1}; x_t]) && \text{(update gate)} \\
 \hat{h}_t &= \tanh(W \cdot [r_t \odot h_{t-1}; x_t]) && \text{(candidate state)} \\
 h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t && \text{(new hidden state)}
 \end{aligned}$$

Compared to LSTM

- **2 gates** instead of 3 → ~25% fewer parameters.
- **No separate cell state**; h_t carries everything.
- **Update gate z_t** plays a combined role of LSTM's f_t and i_t : if $z_t \approx 0$, the state is preserved; if $z_t \approx 1$, it is overwritten.
- **Reset gate r_t** controls how much of the previous state participates in computing the new candidate.

When to prefer

- GRU: small data, faster training, comparable quality.
- LSTM: very long sequences, well-tuned baselines.
- In modern practice, both are mostly replaced by Transformers for NLP. Still excellent for streaming / low-latency settings (speech, time series).

5.4 RNN vs LSTM vs GRU — summary table

	Vanilla RNN	LSTM	GRU
Gates	0	3 (f, i, o)	2 (r, z)
Cell state	none	yes (c_t)	none
Long-range deps	poor (vanishing)	strong	strong
Parameters / layer	$n(m+n+1)$	$4n(m+n+1)$	$3n(m+n+1)$
Training stability	hard	good	good
Common use today	none	sequence tasks where Transformer is overkill	same

6. Seq2Seq + Attention

6.1 The vanilla Seq2Seq problem

For machine translation, summarisation, dialogue, etc., we need a model that reads an input sequence $x_1, \dots, x_S$ and outputs a different sequence $y_1, \dots, y_T$ whose length may differ.

The 2014 architecture (Sutskever et al.; Cho et al.)

- **Encoder RNN.** Reads input tokens left to right; the *final* hidden state h_S^{enc} is a fixed-size “thought vector” that summarises the whole input.
- **Decoder RNN.** Initialised with h_S^{enc} . At each step it produces the next output token, conditioned on its own previous hidden state and previous output token.

The bottleneck problem

All information about the source must squeeze through one vector h_S^{enc} . For a 100-token English sentence and a 1024-dim hidden state, that’s a *severe* compression. Empirically, Seq2Seq without attention

degrades sharply on long sentences (Bahdanau et al. observed this; BLEU dropped from 30 to 15 on 50+ word sentences).

6.2 Bahdanau attention (Bahdanau, Cho, Bengio, 2015)

Idea

At each decoder step t , look at *all* encoder hidden states and decide which ones are relevant for producing the next output token. Mix them weighted by relevance into a **context vector** c_t that is fresh every step.

Equations

For decoder step t , with encoder states $h_1, \dots, h_S$:

```
e_{t, j} = v^T \cdot \tanh( W_s \cdot s_{t-1} + W_h \cdot h_j )      (alignment score, scalar)
\alpha_{t, j} = \exp(e_{t, j}) / \sum_k \exp(e_{t, k})        (softmax over source positions)
c_t = \sum_j \alpha_{t, j} \cdot h_j                          (context vector for step t)
s_t = RNN_decoder( s_{t-1}, [y_{t-1}; c_t] )                (decoder update)
P(y_t | y_{<t}, x) = \text{softmax}( W_o \cdot [s_t; c_t; e(y_{t-1})] ).
```

- s_{t-1} is the decoder hidden state from the previous step (the **query**).
- h_j is the encoder state at source position j (the **key**, also the **value**).
- v, W_s, W_h are learnable.
- “Additive” attention because the inside of $\tanh$ is a sum.

Why the previous state s_{t-1} (not s_t)?

Notice c_t is computed *before* s_t . This lets the decoder use the same c_t *inside* the recurrence that computes s_t . Luong’s variant uses s_t directly — equivalent in expressiveness, slightly different gradient flow.

What attention gives us

1. **No bottleneck.** Every decoder step has direct access to all source positions.
2. **Soft alignment.** The matrix $\alpha \in \mathbb{R}^{T \times S}$ is a soft alignment — much more interpretable than the black box of vanilla Seq2Seq.
3. **Length robustness.** BLEU on long sentences recovered to ~30 in Bahdanau’s experiments.
4. **No one-to-one constraint** (Q1.8). Two decoder tokens can attend to the same source token; one source token can be ignored. Attention is many-to-many.

Worked example

Translate “I love you” → “Je t’aime”. Suppose encoder produces 3 hidden states $h_I, h_{\text{love}}, h_{\text{you}} \in \mathbb{R}^4$. At decoder step 1 (generating “Je”), the alignment scores might be:

```
e_{1,I} = 4.2, e_{1,love} = 1.1, e_{1,you} = 0.2
\alpha_{1,\cdot} = \text{softmax} = [0.94, 0.04, 0.02]
c_1 = 0.94 h_I + 0.04 h_{\text{love}} + 0.02 h_{\text{you}} \approx h_I.
```

So “Je” mostly attends to “I”. At step 2 generating “t”, attention shifts to `h_you` (because “t” is the object pronoun “you”). At step 3 generating “aime”, attention focuses on `h_love`. The attention matrix α reveals this alignment to a human inspector.

6.3 Luong attention (Luong, Pham, Manning, 2015)

Three scoring functions

```
score(s_t, h_j) = s_t^T · h_j           ← dot
                 = s_t^T · W · h_j      ← general (with learnable W)
                 = v^T · tanh(W_a · [s_t; h_j]) ← concat (similar to Bahdanau)
```

Two flavours

- **Global attention:** all encoder positions (most common).
- **Local attention:** a window of size $2D+1$ around an *aligned* position `p_t`. The aligned position is predicted from `s_t`. Reduces compute for very long inputs.

Compared to Bahdanau

	Bahdanau	Luong
Decoder state	<code>s_{t-1}</code> (previous)	<code>s_t</code> (current)
Score	additive only	dot / general / concat
Input to decoder	concat <code>[y_{t-1}; c_t]</code> before RNN	<code>c_t</code> mixed <i>after</i> RNN: $\tilde{s}_t = \tanh(W_c [c_t; s_t])$

Both produce strong translations; modern code typically uses the simpler dot-product form (which we will see again in self-attention).

6.4 Why attention was a watershed

- **Architecturally**, it generalises immediately to non-RNN encoders. If we replace encoder hidden states with the outputs of any sequence model (or even a CNN feature map), attention still works.
- **It is the seed of self-attention.** Replace the encoder by the *same sequence* the decoder is producing, and you get self-attention.
- **It is interpretable.** Alignment plots became the standard diagnostic.

7. The Transformer

7.1 Why Transformer?

RNN/LSTM training is **inherently sequential**: you must compute `h_t` before `h_{t+1}`, ruining GPU parallelism along the time axis. Attention, by contrast, is a single tensor operation that processes all

positions at once. The Transformer (Vaswani et al., 2017, *Attention Is All You Need*) discards recurrence entirely and uses attention everywhere.

7.2 Scaled dot-product self-attention — derivation and intuition

Setup

Input $X \in \mathbb{R}^{T \times d}$ where T is sequence length and d is model dimension. Project to three different spaces:

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V, \quad \text{all} \in \mathbb{R}^{T \times d_k}.$$

- Q ("queries"): "what is each position looking for?"
- K ("keys"): "what does each position offer?"
- V ("values"): "what does each position actually contribute?"

The attention computation

$$\begin{aligned} \text{scores} &= Q \cdot K^T && \in \mathbb{R}^{T \times T} && \text{(raw similarity)} \\ A &= \text{softmax}(\text{scores} / \sqrt{d_k}) && \in \mathbb{R}^{T \times T} && \text{(attention weights, row-stochastic)} \\ \text{output} &= A \cdot V && \in \mathbb{R}^{T \times d_k} \end{aligned}$$

Element $(1, t)$ of A is "how much does position 1 attend to position t ?". Each row of A sums to 1.

Per-position formula

The output at position 1 is:

$$a_1 = \sum_{t=1}^T \alpha_{1, t} \cdot v_t, \quad \text{where } \alpha_{1, t} = \text{softmax}_t(q_1 \cdot k_t / \sqrt{d_k}).$$

This is exactly the structure of Bahdanau attention but with (q_1, k_t, v_t) instead of (s_{t-1}, h_j, h_j) . **The output is a weighted sum of the value vectors** — that is the answer to model paper Q1.12.

Why divide by $\sqrt{d_k}$?

Assume entries of q, k are independent zero-mean unit-variance. Then $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance d_k . As d_k grows, the dot products grow as $\sqrt{d_k}$, pushing the softmax into very peaky regions where gradients vanish. Dividing by $\sqrt{d_k}$ keeps the pre-softmax variance ≈ 1 .

Why is it permutation-equivariant?

If we permute the rows of X by permutation π , then Q, K, V permute identically, and the output permutes too. **There is no notion of position** unless we add one — that's positional encoding (next subsection).

Worked numerical example

Take $T = 3, d_k = 2$. Let:

```

Q = [[1, 0], [0, 1], [1, 1]]
K = [[1, 0], [0, 1], [1, 1]]
V = [[1, 0], [0, 1], [1, 1]]

scores = Q K^T = [[1, 0, 1], [0, 1, 1], [1, 1, 2]]
scaled = scores / √2 ≈ [[0.71, 0, 0.71], [0, 0.71, 0.71], [0.71, 0.71, 1.41]]
A_row_1 = softmax([0.71, 0, 0.71]) ≈ [0.39, 0.19, 0.42]
output_row_1 = 0.39·v_1 + 0.19·v_2 + 0.42·v_3 = [0.81, 0.61]

```

Position 1's output is a mix dominated by `v_1` and `v_3` (which have the highest similarity to `q_1`).

7.3 Multi-Head Attention (MHA)

Motivation

A single softmax forces each head to focus on *one* dominant relationship. But a sentence has many simultaneous relationships: subject–verb agreement, modifier–noun coherence, coreference, syntactic role. MHA runs `h` independent attention computations in parallel, each in a smaller subspace, and lets each head specialise.

Equations

For `i = 1, ..., h`:

$$\text{head}_i = \text{Attention}(X \cdot W_Q^i, X \cdot W_K^i, X \cdot W_V^i) \quad W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{d \times d_k}$$

where `d_k = d / h`. Concatenate and project:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O \quad W_O \in \mathbb{R}^{d \times d}.$$

Why `d_k = d / h`?

So total compute and memory match a single-head attention with dimension `d`. Multi-head is “the same budget, redistributed across heads”.

Final answer formula (Q4.1)

“The outputs of these parallel heads are then concatenated and passed through a linear projection to form the final combined output.” Each head can attend to a different representation subspace.

What heads learn (empirical)

- Some heads track positional offsets (e.g. “attend to the previous token”).
- Some heads track syntactic dependencies (subject–verb).
- Many heads are redundant — Voita et al. (2019) showed up to 80% can be pruned with small loss.

7.4 Positional encoding (PE)

The need

Self-attention is permutation-equivariant. Without positional info, “the cat ate the dog” and “the dog ate the cat” produce identical outputs.

Sinusoidal PE (original Transformer)

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{\{2i/d\}}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{\{2i/d\}}) \end{aligned}$$

- Dimensions are paired (sin/cos). Each pair encodes a different frequency.
- Low-index dimensions oscillate quickly (capture fine position); high-index dimensions oscillate slowly (capture coarse position).
- **Property:** for any fixed offset k , $\text{PE}(\text{pos} + k)$ is a linear function of $\text{PE}(\text{pos})$ — the model can in principle learn relative offsets.

Added (not concatenated) to the token embedding x of each position.

Learned absolute PE (BERT, GPT-2)

A learned $\mathbb{R}^{L \times d}$ lookup table — same dim as the embeddings, learned end-to-end. Simpler, slightly stronger on in-distribution lengths, but **does not extrapolate** beyond the training maximum length.

Rotary PE (RoPE — Su et al., 2021; used in LLaMA, GPT-NeoX)

Rotate the (q, k) pairs by an angle proportional to position. The dot product $q_i \cdot k_j$ then depends only on the *relative offset* $i - j$. This makes RoPE both relative and supports moderate length extrapolation.

ALiBi (Press et al., 2022)

Add a linear bias $-m \cdot |i - j|$ to attention scores; different heads use different slopes m . No learnable parameters, excellent extrapolation, cheap. Used in BLOOM.

Decoder-only models can be trained without PE (Q1.2)

With a *causal* attention mask, position 1 can attend to positions $1, \dots, 1$ — *the count of positions attended to is itself positional information*. The model can extract position from the mask alone. Encoder-only models can't, because attention is bidirectional and provides no positional asymmetry.

7.5 Masked self-attention (decoder)

The constraint

During teacher-forced training, we feed the entire target sequence at once for efficiency. But the prediction at position 1 must depend only on positions ≤ 1 (auto-regressive property), else at inference the model would expect to peek at future tokens it hasn't generated.

The mask

Set pre-softmax score $e_{\{1, t\}} = -\infty$ for $t > 1$. After softmax, those weights are 0. As a matrix:

```
mask (T=4) =
[[ 0, -∞, -∞, -∞],
 [ 0,  0, -∞, -∞],
 [ 0,  0,  0, -∞],
 [ 0,  0,  0,  0]]
```

Add this mask to the scaled scores before softmax. Position 1 attends only to 1, ..., 1.

Worked detail (Q4.2)

“Why is masking required and how is it applied?” Answer: to preserve the auto-regressive property; mathematically, set $e_{\{1, t\}} = -\infty \forall t > 1$ before softmax, so attention weights to future tokens become zero. This ensures training and inference are consistent.

7.6 The full Transformer block

Position-wise feed-forward network (FFN)

After (multi-head) attention, each position’s vector is passed through the *same* two-layer MLP independently:

```
FFN(x) = max(0, x · W_1 + b_1) · W_2 + b_2,    W_1 ∈ R^{d × 4d}, W_2 ∈ R^{4d × d}.
```

The expansion ratio is typically 4 (so a $d = 768$ model has an FFN with hidden size 3072). Modern variants use **GELU** instead of ReLU, and **SwiGLU** in LLaMA.

The FFN provides per-position non-linearity and is where most of the *world knowledge* of a Transformer is stored — attention mixes information; FFN transforms it.

Residual connections and Layer Norm

Post-LN (original Transformer):

```
x_attn = LayerNorm( x + MHA(x) )
x_out  = LayerNorm( x_attn + FFN(x_attn) )
```

Pre-LN (modern, more stable for deep models):

```
x_attn = x + MHA( LN(x) )
x_out  = x_attn + FFN( LN(x_attn) )
```

Pre-LN avoids exploding activations and is used in GPT-2/3, LLaMA, etc.

LayerNorm (the per-token normalisation)

For a vector $x \in \mathbb{R}^d$:

$$\begin{aligned}\mu &= (1/d) \sum x_i \\ \sigma^2 &= (1/d) \sum (x_i - \mu)^2 \\ \hat{x} &= (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} \\ y &= \gamma \odot \hat{x} + \beta.\end{aligned}$$

Unlike BatchNorm, LayerNorm uses *no batch-level statistics*, so it is sequence-length-independent and works identically at training and inference.

The encoder block (one Transformer layer)

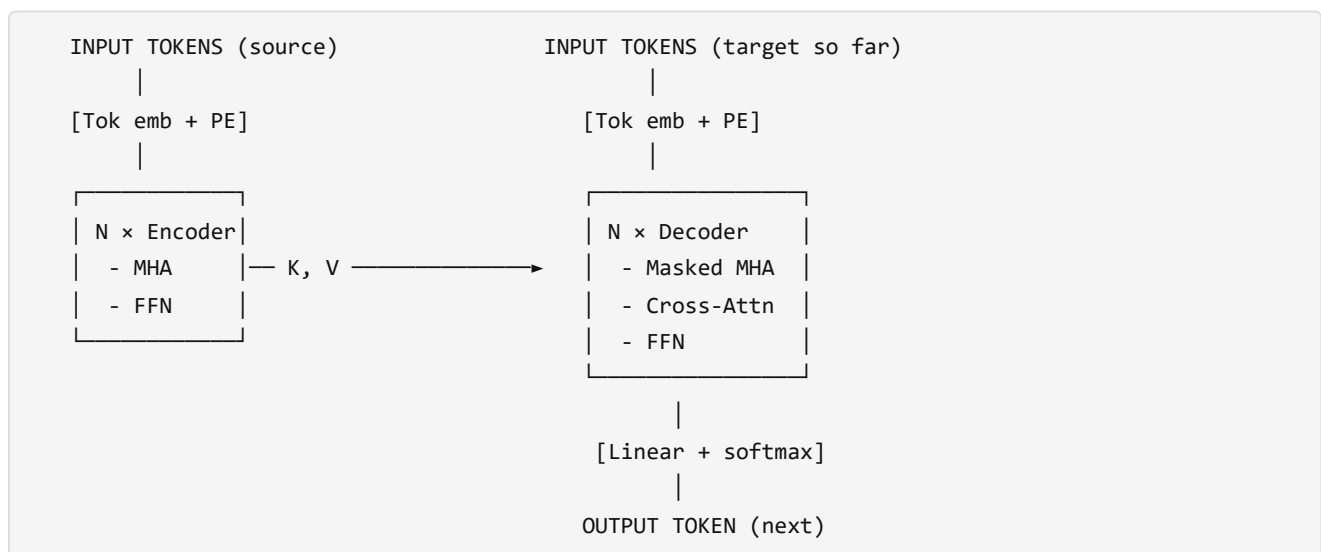
1. MHA(x) + residual + LN → contextualised token representations
2. FFN(x) + residual + LN → per-token non-linear transformation

The decoder block

Three sub-layers:

1. Masked MHA(self-attn over decoder) + residual + LN
2. MHA(Q from decoder, K,V from encoder) ← cross-attention + residual + LN
3. FFN + residual + LN

Architecture diagram



7.7 Complexity

Operation	Time	Memory
Self-attention	$O(T^2 \cdot d)$	$O(T^2)$
FFN	$O(T \cdot d^2)$	$O(T \cdot d)$
Total per layer	$O(T^2 \cdot d + T \cdot d^2)$	dominated by attention for long sequences

For $T = 2048$, $d = 768$, $T^2 \cdot d = 3.2$ G ops, $T \cdot d^2 = 1.2$ G ops — attention dominates. This is the motivation for **FlashAttention** (memory-efficient implementation), **sparse / local attention** (Longformer,

BigBird), and **linear attention** (Performer).

Per-token inference latency (Q1.9)

With **KV-caching** (cache keys and values from previous tokens, only compute new query), each new token costs $O(T \cdot d)$ for attention. Vanilla RNN is $O(d)$ per token (state update only). So **decoder-only Transformer is NOT lower-latency per token at inference**; the statement in the model paper Q1.9 is False. (Training is a different story: Transformer parallelises across all positions in a single matmul; RNN cannot.)

8. Encoder vs Decoder vs Encoder-Decoder

8.1 The three families

1. **Encoder-only** (BERT, RoBERTa). Bidirectional self-attention. No generation. Output = contextual embedding per token. Use for: classification, NER, extractive QA, similarity.
2. **Decoder-only** (GPT, LLaMA). Causal self-attention. Auto-regressive generation. Use for: chat, code, creative text, summarisation (with prompt), translation (with prompt).
3. **Encoder-decoder** (T5, BART, original Transformer). Encoder reads input, decoder generates output with cross-attention. Use for: translation, summarisation, structured seq2seq.

8.2 BERT — Masked Language Modelling

Pre-training objective (MLM)

1. Sample 15% of tokens.
2. For those:
 - 80% replace with [MASK].
 - 10% replace with a *random* token.
 - 10% leave unchanged.
3. Predict the original token at every chosen position.

Why the 80/10/10 split? If we always used [MASK], the model would learn that [MASK] is a special signal — useless at fine-tuning time when [MASK] doesn't appear. The 10% random + 10% unchanged force the model to maintain a useful representation at *every* position, not just at masked ones.

NSP (Next Sentence Prediction)

Binary classification on the special [CLS] token: is sentence B the actual next sentence after A, or a random sentence? Later papers (RoBERTa) found NSP doesn't help and removed it.

Architecture

- BERT-base: $L=12$ layers, $h=12$ heads, $d=768$, $\approx 110M$ params.
- BERT-large: $L=24$, $h=16$, $d=1024$, $\approx 340M$ params.
- Input: [CLS] sent_A [SEP] sent_B [SEP], with **segment embeddings** ($A=0$, $B=1$) and **learned positional embeddings** added to token embeddings.

Fine-tuning

Add one linear head on [CLS] for classification, or per-token heads for tagging tasks. Train end-to-end on the labelled task.

8.3 GPT — Causal LM

Pre-training objective

$L = - \sum_t \log P(w_t \mid w_1, \dots, w_{\{t-1\}})$. That's it. Predict the next token, left to right.

Architecture

Stack of decoder blocks (masked MHA + FFN; *no cross-attention* because there's no encoder). Output = next-token distribution.

Generation

At inference, sample auto-regressively: $w_t \sim P(\cdot \mid w_{\{<t\}})$. Use temperature, top-k, top-p (nucleus), or beam search for higher-quality outputs.

Strengths and weaknesses vs BERT

- Generation: GPT wins (BERT can't generate well).
- Classification: BERT historically wins (bidirectional context); GPT can match with enough scale.

8.4 T5 — Text-to-Text Transfer Transformer (Raffel et al., 2020)

Idea

Everything is text-in / text-out. Translation, summarisation, classification, QA — all framed as “input prompt → output string”.

Pre-training: span corruption

Replace random spans with sentinel tokens; the target is the sentinels + their content:

```
Input: Thank you <X> me to your party <Y> week.  
Target: <X> for inviting <Y> last <Z>
```

The decoder learns to reproduce the deleted spans, delimited by sentinels. This is a *generalisation* of MLM (multi-token spans) and a *specialisation* of BART (just one noise type).

Sizes

T5-small → T5-XL (3B) → T5-XXL (11B). The T5 paper systematically studied design choices (architecture, objective, data, length).

8.5 BART — Bidirectional and Auto-Regressive Transformer (Lewis et al., 2020)

Idea

Combine BERT-style bidirectional encoder with GPT-style auto-regressive decoder, train on a *denoising* objective.

Noise functions (use any mix)

1. **Token masking** (BERT-style).
2. **Token deletion** (model must figure out where deletions happened).
3. **Text infilling** (replace a span with a *single* [MASK] — model must guess the length too).
4. **Sentence permutation** (reorder).
5. **Document rotation** (rotate so a random token is the new start).

The combination is what makes BART specifically strong at summarisation: text-infilling matches the abstractive nature of summary writing.

8.6 Choosing the right family

Task	Recommended family
Sentiment / topic classification	encoder-only (BERT)
NER, POS tagging	encoder-only (BERT)
Extractive QA (SQuAD)	encoder-only
Abstractive summarisation	encoder–decoder (BART, T5) or decoder-only (GPT)
Translation	encoder–decoder OR decoder-only (Q1.20: D)
Open-ended generation, chat	decoder-only (GPT, LLaMA)
Code generation	decoder-only
Sentence embedding / similarity	encoder-only (Sentence-BERT)

9. Pre-training and Fine-tuning

9.1 The two-stage paradigm

1. **Pre-training.** Self-supervised on a huge unlabelled corpus (CommonCrawl, books, code). The model learns general language structure, world knowledge, grammar.
2. **Fine-tuning.** Supervised on a small task-specific dataset (often 1k–100k examples). The model adapts pre-trained weights to the downstream task.

This works because most of the *useful representational capacity* is learned during pre-training; fine-tuning just steers it.

9.2 Why base LMs fail at zero-shot instruction following (Q1.13)

A base LM's training corpus is overwhelmingly *not* in (instruction, response) format. Internet text is articles, conversations, code, etc. — none of which look like:

```
Instruction: Write a haiku about autumn.  
Response: Crimson leaves descend...
```

When you give a base LM the instruction "Write a haiku about autumn.", it often *continues* the instruction (because that's what its training data looks like — text continuations), rather than *answering* it.

Solution: instruction tuning (Section 11).

9.3 Parameter-Efficient Fine-Tuning (PEFT)

Full fine-tuning is expensive

A 7B parameter LM takes ~28 GB of weights in FP32, plus ~28 GB for gradients, plus ~56 GB for Adam optimiser states. Total ~112 GB — does not fit on commodity GPUs.

Adapter layers (Houlsby et al., 2019)

Insert a small bottleneck MLP into each Transformer block:

```
adapter(h) = h + W_up · ReLU( W_down · h ),    W_down ∈ ℝ^{d × r}, W_up ∈ ℝ^{r × d}.
```

With $r = 8 \ll d$, you add ~0.5% extra params. Train *only* the adapters; freeze the rest.

LoRA — Low-Rank Adaptation (Hu et al., 2021)

Instead of inserting new layers, *additively decompose* the weight update of an existing linear layer:

```
W_new = W_frozen + ΔW,    where ΔW = B · A,    B ∈ ℝ^{d × r}, A ∈ ℝ^{r × d}.
```

- Train only A, B . W_{frozen} stays untouched.
- At inference, merge: $W \leftarrow W_{\text{frozen}} + B \cdot A$. **No latency overhead.**
- Typical $r = 8$ for a $d = 4096$ model $\rightarrow$ 0.05% trainable params.

QLoRA (Dettmers et al., 2023)

- Quantise W_{frozen} to **4-bit NF4** (NormalFloat4).
- LoRA adapters stay in 16-bit.
- Memory drops 4×. Enables fine-tuning of 65B models on a single 48GB GPU.

Prefix / Prompt tuning

- **Prompt tuning** (Lester et al.): prepend k learnable "soft" embeddings to the input. Train only these. Works at very large scales ($>10\text{B}$ params).

- **Prefix tuning** (Li & Liang): inject learnable prefixes into the K, V of *every* attention layer. Stronger than prompt tuning at smaller scales.

Comparison

	Full FT	LoRA	QLoRA	Prompt tuning
Trainable	100%	0.05–1%	0.05–1%	<0.1%
Memory	full	small	tiny	tiny
Quality at 7B	best	matches	matches	slightly worse
Quality at 70B+	best	matches	matches	matches
Inference cost	same	same (merged)	same	extra prompt tokens

10. Prompting and In-context Learning

10.1 The prompting paradigm

At sufficient scale (~10B+), LMs do not need fine-tuning to perform many tasks. You just *prompt* them with a description and optionally a few examples.

Zero-shot vs few-shot

ZERO-SHOT:

Translate to French: "I love you" →

FEW-SHOT (3 examples):

English: I love you. French: Je t'aime.

English: Good morning. French: Bonjour.

English: Thank you. French: Merci.

English: How are you? French:

Why few-shot works (in-context learning, ICL)

A still-debated phenomenon. Theories: 1. **Implicit Bayesian inference**. The prompt induces a posterior over latent tasks; the LM reasons over this posterior. 2. **Attention as gradient descent**. Some work (von Oswald et al., 2022) shows linear self-attention on (input, output) pairs literally performs one step of gradient descent on a least-squares problem. 3. **Concept retrieval**. The prompt activates the right "skill" already learned during pre-training.

No consensus, but the capability is robust enough to be the dominant LLM interface.

10.2 Chain-of-Thought (CoT) prompting (Wei et al., 2022)

The trick

Include the *reasoning chain* in the few-shot examples:

Q: Roger has 5 tennis balls. He buys 2 cans, each with 3. How many balls now?
A: Roger started with 5 balls. 2 cans $\times$ 3 balls each = 6 new balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half are golf balls, and half of the golf balls are blue. How many blue golf balls?
A:

The model then produces its own reasoning + answer.

Why CoT helps

- Multi-step problems benefit from *spending compute on intermediate tokens*. Each generated token is another forward pass.
- The model commits to intermediate decisions as text, which constrains subsequent ones.
- Emergent: CoT only helps reliably at ~60B+ params.

Zero-shot CoT (Kojima et al.)

Just append “Let’s think step by step.” to the prompt. Surprisingly strong. Works because the model has seen many “Let’s think step by step...” reasoning chains in the training corpus.

Self-consistency (Wang et al.)

Sample N independent CoT chains (with non-zero temperature). Take a majority vote on the final answer. Often gives 5–15 point gains on arithmetic benchmarks.

10.3 Prompt engineering vs prompt tuning vs prefix tuning

Method	Who writes it?	Continuous or discrete?	Where does it sit?
Prompt engineering	human	discrete tokens (natural language)	prepended to input
Prompt tuning	learned	continuous vectors (no token mapping)	prepended to input only
Prefix tuning	learned	continuous vectors	injected as K, V at <i>every</i> attention layer

Prompt engineering is what most practitioners do day-to-day. Prompt/prefix tuning are training methods — usually deployed only when you cannot fine-tune full weights.

11. Instruction Tuning and Synthetic Data

11.1 Supervised Fine-Tuning (SFT) for instructions

Take (instruction, response) pairs and minimise next-token cross-entropy on the response:

$$L_{\text{SFT}} = - \sum_{\{(x, y) \in D\}} \sum_t \log \pi_{\theta}(y_t \mid x, y_{\{<t\}}).$$

After SFT, the model defaults to instruction → answer behaviour. SFT is **step 1** of the InstructGPT / ChatGPT pipeline.

11.2 Where do instructions come from?

Human-curated (FLAN, Super-NaturalInstructions)

Take existing NLP datasets, write prompt templates that frame them as instructions, mix together. Quality is high, but coverage is limited to existing tasks.

Synthetic — four named methods you must know

Self-Instruct (Wang et al., 2022)

1. Start with a seed pool of ~175 human-written tasks.
2. Sample 6 examples from the pool and ask the LLM to “Write 8 more diverse tasks following this style.”
3. Classify whether each task is classification or generation.
4. Generate responses (input-first or output-first depending on classification).
5. **Filter** duplicates, low-quality, and degenerate outputs.
6. Add survivors back to the pool. Repeat.
7. Fine-tune the LLM on the final pool of ~52k instruction-response pairs.

Boosted GPT-3’s instruction-following to nearly the level of InstructGPT, at a fraction of the cost.

Evol-Instruct (WizardLM, Xu et al., 2023) — Q1.14

Starts with simple instructions and *evolves* them to harder versions. Operations applied by the “Instruction Evolver”: - **Add constraints** (“Solve this problem, but show your reasoning in fewer than 20 words.”). - **Deepen** (“Explain at a graduate level instead of high school.”). - **Concretise** (“Give a numerical example with specific values.”). - **Increase reasoning steps** (“Explain why, step by step.”). - **Complicate input** (“Now do the same task with this much longer input...”). - **In-breadth** (generate a completely new task in a different domain).

Each round produces a harder version; the LLM generates responses; filtering removes failures (responses that are off-topic, evasive, or syntactically broken). Result: WizardLM, which beat earlier IFT models on hard reasoning benchmarks.

Orca (Mukherjee et al., 2023)

Train a smaller model on **explanations** from a strong teacher (GPT-4), not just final answers. The teacher is prompted with system messages like “Explain like I’m five” or “Provide step-by-step reasoning”. The student learns *reasoning traces* and generalises better than mere imitation of answers.

Instruction Back-translation (Humpback, Li et al., 2023)

Inverts the usual direction. Take a piece of unlabelled web text and treat it as a *response*; ask an LLM to invent a plausible *instruction* that would produce it. Filter to keep only high-quality (instruction, response)

pairs. Then SFT.

Why this works: the response is real text from the wild, ensuring style diversity that synthetic responses lack.

11.3 Why SFT is not enough → motivates RLHF (Q5)

SFT teaches the *form* of instruction following. But for most prompts, many valid responses exist; SFT cannot encode *human preferences over those responses* — helpful vs unhelpful, harmless vs harmful, honest vs sycophantic.

Concrete failure (model paper Q5). Prompt: “What’s the best way to lose weight quickly?” An SFT model trained on web text might helpfully suggest “Stop eating entirely” — syntactically perfect, on-topic, and dangerous. We need a signal that *ranks* responses by quality. Humans provide that via *pairwise preferences*, leading to a **reward model** and **RLHF** (Section 15).

12. RL fundamentals (as applied to LLMs)

12.1 Markov Decision Process (MDP) for an LLM

- **State** s_t = the prompt x plus all tokens generated so far $y_{\{<t\}}$. (Model paper Q6.)
- **Action** a_t = next output token, from vocabulary V .
- **Transition** = deterministic in this case: appending the action to the state.
- **Reward** $r(x, y)$ = scalar from the reward model. Usually given at the **end** of generation (sparse).
- **Policy** $\pi_\theta(a \mid s_t)$ = the LLM’s softmax distribution over the vocabulary.

The “Environment” in LLM-RL (Q1.15)

Not OpenAI Gym. Not the Transformer. The environment is the **abstract loop of (text input → model output → reward)**: the model emits tokens, the environment accepts them as a generation, and at the end the reward model assigns a score. This is the answer B in Q1.15.

12.2 Returns and discount factor

The return from time t is:

$$G_t = r_{\{t+1\}} + \gamma r_{\{t+2\}} + \gamma^2 r_{\{t+3\}} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{\{t+k+1\}}.$$

- $\gamma \in [0, 1]$ is the discount factor.
- For LLMs, rewards are typically end-of-sequence, so $G_t = \gamma^{T-t-1} \cdot r_{\text{final}}$ (almost no discounting in practice).

12.3 Value functions

State-value $V^\pi(s)$: expected return starting from state s and following policy π :

$$V^{\pi}(s) = E_{\pi} [G_t \mid s_t = s].$$

Action-value $Q^{\pi}(s, a)$: expected return after taking action a in state s , then following π :

$$Q^{\pi}(s, a) = E_{\pi} [G_t \mid s_t = s, a_t = a].$$

Advantage $A^{\pi}(s, a)$: how much better is action a than the policy's average action at state s :

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

Why advantage? Subtracting V is a variance-reducing baseline — it doesn't change the gradient's expectation but tightens its variance enormously.

12.4 Generalised Advantage Estimation (GAE)

Given a learned value function $\hat{V}$ (the **critic**) and the TD residual:

$$\delta_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t),$$

GAE estimates the advantage as a geometric average of n -step TD errors:

$$\hat{A}_t^{\{GAE(\gamma, \lambda)\}} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}.$$

- $\lambda = 0$: pure TD(0), low variance, high bias.
- $\lambda = 1$: Monte-Carlo return $-\hat{V}(s_t)$, low bias, high variance.
- Typical $\lambda = 0.95$.

GAE is the de-facto advantage estimator inside PPO.

13. Reward modelling — Bradley–Terry

13.1 The data

Humans rank two responses (y_1, y_2) to the same prompt x . Annotation: “ y_w is preferred to y_l ” $\rightarrow$ labelled tuple (x, y_w, y_l) .

A typical RLHF dataset has 10k–500k such pairs.

13.2 The Bradley–Terry model

Posit a latent scalar score $r_{\phi}(x, y)$ for each (x, y) . Define:

$$P(y_1 > y_2 \mid x) = \sigma(r_{\phi}(x, y_1) - r_{\phi}(x, y_2)), \quad \sigma(z) = 1 / (1 + e^{-z}).$$

This is a logistic regression on the *score difference*. Intuition: only the difference matters; absolute scores have a free constant.

Derivation of the training loss

Likelihood of one preference observation:

$$P(y_w > y_l \mid x; \phi) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

Negative log-likelihood (the **reward-model loss**):

$$L(\phi) = -E_{\{(x, y_w, y_l) \sim D\}} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

Equivalently we MAXIMISE $\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$ — which is exactly Q1.16 option B in the model paper.

What does the gradient look like?

$$\nabla_\phi \log \sigma(r_w - r_l) = (1 - \sigma(r_w - r_l)) \cdot \nabla_\phi (r_w - r_l).$$

- If the model already ranks y_w above y_l (the sigmoid argument is large), the gradient is tiny — don't fix what isn't broken.
- If the model ranks them wrong, the gradient is large and pushes r_w up, r_l down.

This is a soft hinge-like loss; the closer to the wrong side, the stronger the update.

13.3 Implementation

- Initialise the reward model from the SFT model.
- Replace the LM head with a single scalar output head.
- Forward the full sequence $[x, y]$, pool the final token's hidden state, project to scalar.
- Train on preference pairs with the BT loss.

13.4 Pitfalls

Reward hacking

The policy can find outputs that score high on r_ϕ but are bad to actual humans (e.g. extra-long, repetitive, sycophantic, or laden with confident-sounding nonsense). This is *Goodhart's Law* applied to a learned proxy. Mitigations: KL regularisation, periodic reward-model refresh on new policy outputs, multiple reward models.

Annotator disagreement

Humans disagree on subjective prompts. BT effectively averages preferences across annotators — bias-prone. Best practice: multi-annotator agreement filtering, calibration sets, demographic-aware sampling.

Distribution shift

The reward model is trained on outputs from the SFT model. As RLHF progresses, the policy's outputs drift; the RM may become miscalibrated on the new distribution. Solutions: rejection sampling + reward model retraining ("iterated RLHF").

14. Policy gradients, REINFORCE, PPO

14.1 The policy gradient theorem

Statement

For policy π_θ and trajectory $\tau = (s_0, a_0, s_1, a_1, \dots)$:

$$\nabla_\theta J(\theta) = E_{\{\tau \sim \pi_\theta\}} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right].$$

You can replace G_t with $Q^\pi(s_t, a_t)$ or $A^\pi(s_t, a_t)$ (with a baseline) without changing the expectation, only the variance.

Derivation sketch

Start from $J(\theta) = E_{\{\tau \sim \pi_\theta\}}[R(\tau)]$ and apply the **log-derivative trick**:

$$\nabla_\theta E_{\{x \sim p_\theta\}}[f(x)] = E_{\{x \sim p_\theta\}}[f(x) \cdot \nabla_\theta \log p_\theta(x)].$$

For a trajectory, $\log p_\theta(\tau) = \sum_t \log \pi_\theta(a_t | s_t)$ (transitions are θ -independent). Differentiate and you get the policy gradient.

14.2 REINFORCE (Williams, 1992)

Monte-Carlo policy gradient. Sample one rollout per gradient update:

$$\nabla_\theta J \approx \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot (G_t - b(s_t)).$$

The baseline $b(s_t)$ (typically $\hat{V}(s_t)$) reduces variance without biasing the gradient. **Pros:** simple, unbiased. **Cons:** high variance, on-policy only (each rollout used once), sample-inefficient.

14.3 Importance Sampling (IS)

Motivation

We collected rollouts with policy π_{old} , but we have updated to π_θ . Can we still use the data? Yes, with IS:

$$E_{\{x \sim \pi_\theta\}}[f(x)] = E_{\{x \sim \pi_{old}\}} \left[\left(\pi_\theta(x) / \pi_{old}(x) \right) \cdot f(x) \right].$$

The ratio $\rho = \pi_\theta / \pi_{old}$ reweights the samples. The estimator is unbiased.

The danger

If π_θ puts large mass where π_{old} puts tiny mass, ρ can be huge $\rightarrow$ variance explodes. This is why PPO **clips** the ratio.

14.4 PPO and PPO-CLIP (Schulman et al., 2017)

The surrogate objective

Let $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{old}(a_t | s_t)$ be the per-step IS ratio. The PPO-CLIP objective is:

$$L^{\{CLIP\}}(\theta) = E_t [\min(r_t \cdot \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \cdot \hat{A}_t)].$$

Here $\text{clip}(x, a, b) = \max(a, \min(b, x))$ and $\epsilon \approx 0.1-0.2$.

What the clip does (case analysis)

Case 1: $\hat{A}_t > 0$ (good action). We want to increase $\pi_\theta(a_t | s_t)$. - If $r_t < 1 + \epsilon$, the clip is inactive: gradient ascends $r_t \cdot \hat{A}_t$ normally. - If $r_t > 1 + \epsilon$, the clipped term $(1 + \epsilon) \cdot \hat{A}_t$ is constant w.r.t. $\theta \rightarrow$ no gradient. The $\min$ picks the smaller objective, which is the clipped one. **Update stops** — we don't push the probability further.

Case 2: $\hat{A}_t < 0$ (bad action). We want to decrease $\pi_\theta(a_t | s_t)$. - If $r_t > 1 - \epsilon$, normal gradient. - If $r_t < 1 - \epsilon$, $(1 - \epsilon) \cdot \hat{A}_t$ is larger (less negative) $\rightarrow$ clip inactive, gradient continues — we keep decreasing the prob.

The $\min$ ensures we get a **pessimistic lower bound**: improvements outside the trust region are clipped; deteriorations are not. This is what makes PPO safe to run for multiple epochs over the same data.

Full PPO loss

$$L = L^{\{CLIP\}} - c_1 \cdot (\hat{V}(s_t) - V_{target})^2 + c_2 \cdot H[\pi_\theta(\cdot | s_t)].$$

- $c_1 \approx 0.5$: value-function loss weight.
- $c_2 \approx 0.01$: entropy bonus — encourages exploration.

Algorithm sketch

```
for iteration = 1, 2, ...:
    Sample N trajectories with  $\pi_{\theta\_old}$ .
    Compute  $\hat{A}_t$  (using GAE).
    For K epochs:
        For each minibatch:
            Compute  $L^{CLIP} + \text{value loss} + \text{entropy bonus}$ .
            SGD step on  $\theta$ .
     $\pi_{\theta\_old} \leftarrow \pi_\theta$ .
```

Typical hyperparams: $N = 2048$ steps, $K = 10$ epochs, batch size 64.

15. RLHF objective with KL regularisation; Constitutional AI; DPO

15.1 The three-step RLHF pipeline

1. **SFT**. Supervised fine-tune base LM on (instruction, demonstration).
2. **Reward model**. Train RM on pairwise preferences (Bradley–Terry).
3. **RL fine-tune** the SFT policy against RM with KL regularisation to a frozen reference.

15.2 The regularised reward-maximisation objective (Q7)

$$\max_{\theta} \mathbb{E}_{\{x \sim D, y \sim \pi_{\theta}(\cdot | x)\}} [r_{\phi}(x, y)] - \beta \cdot \text{KL}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)).$$

Equivalently, with the KL pulled into the expectation:

$$\mathbb{E}_{\{x \sim D\}} \mathbb{E}_{\{y \sim \pi_{\theta}(\cdot | x)\}} [r_{\phi}(x, y) - \beta \cdot \log(\pi_{\theta}(y | x) / \pi_{\text{ref}}(y | x))].$$

- π_{ref} is the frozen SFT model (the *reference policy*).
- β : regularisation strength. Tune; larger $\rightarrow$ stay closer to SFT.
- KL penalises policy drift; this prevents *reward hacking* and preserves output diversity / fluency.

Per-token implementation

In practice, the KL term is folded into a per-token reward:

$$\tilde{r}_t = r_{\phi}(x, y) \cdot 1[t = T] - \beta \cdot (\log \pi_{\theta}(a_t | s_t) - \log \pi_{\text{ref}}(a_t | s_t)).$$

- The Bradley–Terry reward fires only at the final token (sequence-level).
- Every token receives a small per-token KL penalty.

The augmented per-token reward $\tilde{r}_t$ is then plugged into PPO as the reward signal.

15.3 Why KL regularisation specifically?

- It is the natural divergence on a reference distribution: zero when policies match, asymmetric (penalises putting mass where π_{ref} has none — heavily).
- It has a tractable per-token decomposition for sequential models.
- It corresponds to a Bayesian interpretation: maximise reward subject to staying near a prior π_{ref} .
The closed-form solution $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \cdot \exp(r(x,y)/\beta)$ directly motivates DPO.

15.4 Closed-form optimal policy (used to derive DPO)

Without function-class restrictions, the optimum of the regularised objective is:

$$\pi^*(y | x) = (1 / Z(x)) \cdot \pi_{\text{ref}}(y | x) \cdot \exp(r_{\phi}(x, y) / \beta),$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y | x) \cdot \exp(r_{\phi}(x, y) / \beta)$ is a normaliser. This is the Boltzmann posterior with r_{ϕ} / β as the log-likelihood and π_{ref} as the prior.

15.5 Constitutional AI (Anthropic, Bai et al., 2022)

Motivation

Hand-collecting harmlessness preferences is expensive and slow. CAI replaces this with self-supervised critique driven by a written “constitution” of principles.

SL-CAI (supervised stage)

1. Sample a response from a “helpful-only” model to a potentially harmful prompt.
2. Ask the model: “Critique the previous response based on this principle: .”
3. Ask the model: “Revise the response to address the critique.”
4. Fine-tune on (prompt → revised response) pairs.

RL-CAI (RL stage)

1. Sample multiple responses for each prompt.
2. Ask the model: “Which response is better with respect to ?” → AI-generated preference pairs.
3. Train a reward model on these.
4. PPO with KL regularisation against the SL-CAI model.

Net effect: scale harmlessness training to millions of examples without expensive human labelling.

15.6 DPO — Direct Preference Optimisation (Rafailov et al., 2023)

Motivation

Skip the reward model and PPO entirely. Use the closed-form π^* from Section 15.4 to construct a *supervised* loss on preference pairs.

Derivation sketch

From $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \cdot \exp(r(x,y)/\beta)$, we can invert:

$$r(x, y) = \beta \cdot \log(\pi_{\theta}(y | x) / \pi_{\text{ref}}(y | x)) + \beta \cdot \log Z(x).$$

Plug this into the Bradley–Terry likelihood. The $\log Z(x)$ cancels in the *difference* $r(y_w) - r(y_l)$, leaving:

$$\begin{aligned} L_{\text{DPO}} = & - E_{\{(x, y_w, y_l)\}} \log \sigma(\\ & \beta \cdot [\log(\pi_{\theta}(y_w | x) / \pi_{\text{ref}}(y_w | x)) - \log(\pi_{\theta}(y_l | x) / \pi_{\text{ref}}(y_l | x))] \\ &). \end{aligned}$$

Train this with standard supervised learning. Simpler, more stable, no reward-hacking through a separate model, competitive with PPO-RLHF.

16. Retrieval-Augmented Generation

16.1 Why retrieval?

Parametric memory (the model's weights) is frozen at training. The world changes; models cannot: - Cite sources. - Be updated cheaply. - Recall low-frequency facts.

Retrieval lets a generator pull fresh, verifiable evidence at inference time.

16.2 Retrieval techniques

Sparse retrieval (BM25)

A bag-of-words scoring function with term-frequency saturation and length normalisation:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \left(\text{tf}(t, d) \cdot (k_1 + 1) \right) / \left(\text{tf}(t, d) + k_1 \cdot (1 - b + b \cdot |d|/\text{avgdl}) \right).$$

- IDF up-weights rare terms.
- Length normalisation prevents bias toward long docs.
- Implemented with an inverted index → sub-linear lookup.

Dense retrieval (DPR — Karpukhin et al., 2020)

Two BERT encoders: one for queries, one for passages. Score is $q \cdot d$ (or cosine). Build an **ANN index** (FAISS, Annoy, HNSW) on passage vectors; at query time, embed the query and find top-K nearest neighbours in **sub-linear** time.

Complexity (Q1.10)

For corpus of N docs, both BM25 (inverted index) and dense ANN return top-K in $o(N)$ (sub-linear). Answer: **C**.

Hybrid

BM25 first-stage (recall) + dense rerank (precision). Often combined with a cross-encoder reranker (much more compute, but the candidate set is small).

16.3 RAG flavours — and which use only one document

REALM (Gua et al., 2020)

- A single latent retrieval $z \in \{\text{top-K passages}\}$.
- $P(y \mid x) = \sum_z P(y \mid x, z) \cdot P(z \mid x)$.
- Trained end-to-end with stop-gradient tricks on the retriever.
- **Final output traces back to a single document** (the chosen z).

RAG-Sequence (Lewis et al., 2020)

For each of the top-K passages independently, generate the entire output sequence, then marginalise:

$$P(y \mid x) = \sum_z P(z \mid x) \cdot P(y \mid x, z).$$

Each candidate y corresponds to *one* passage z . **The final returned output uses one document.**

RAG-Token (Lewis et al., 2020)

At every output token, marginalise over passages:

$$P(y_t \mid x, y_{\{<t\}}) = \sum_z P(z \mid x) \cdot P(y_t \mid x, y_{\{<t\}}, z).$$

Different output tokens can be sourced from different passages. **Multiple documents contribute.**

FiD — Fusion-in-Decoder (Izacard & Grave, 2021)

Encode each of the K retrieved passages *separately* (parallel). The decoder cross-attends over the **concatenation of all encoder outputs**, fusing evidence inside the decoder. - Encoding cost: $O(K \cdot L^2)$ (parallel) rather than $O((K \cdot L)^2)$ (concatenated). - Final output can mix multiple documents.

Summary (Q1.3)

"Final output contains evidence only from a single document": - ☒ **REALM**: single latent z . - ☒ **FiD**: fuses across docs. - ☒ **RAG-Token**: per-token marginalisation across docs. - ☒ **RAG-Sequence**: per-output marginalisation; each candidate output is from one doc.

Answer: A and D.

16.4 Open problems

Multi-hop retrieval (Q1.7)

Many real questions need *iterative* retrieval. E.g. "What is the height of the tallest player on the 2022 Real Madrid roster?" requires: 1. Retrieve roster → identify tallest player. 2. Retrieve height info for that player.

A single-shot retriever almost always fails the second hop. Solutions: iterative retrieval (ReAct-style), graph-based retrieval, query decomposition.

Hallucination (Q1.6)

RAG REDUCES hallucination compared to closed-book generation. Why? - The model has direct evidence to ground its claims. - Users can inspect the retrieved passages and trace errors to the retriever. - The retriever can be improved independently.

The opposite claim (Q1.6 = False) is one of the most common student traps.

17. Agentic frameworks

17.1 Non-agentic vs agentic workflows (Q8)

- **Non-agentic (zero-shot).** Single forward pass produces the entire output. No revision. Like writing an essay top-to-bottom without ever editing.
- **Agentic.** Iterative loop: plan → execute → critique → revise. Like writing an essay with outline, research, draft, feedback, revision.

Agentic workflows consistently outperform single-shot generation on multi-step / long-horizon tasks, *with the same base model*.

17.2 ReAct (Yao et al., 2022) — Reasoning + Acting

Interleaves Thoughts (chain-of-thought reasoning), Actions (tool calls), and Observations (tool outputs):

```
Question: Who is the wife of the singer of Bohemian Rhapsody?

Thought 1: I need to find the singer of Bohemian Rhapsody.
Action 1: Search["Bohemian Rhapsody singer"]
Observation 1: Freddie Mercury was the lead singer of Queen.

Thought 2: Now I need Freddie Mercury's wife.
Action 2: Search["Freddie Mercury wife"]
Observation 2: Mercury was never married; his long-term partner was Mary Austin.

Thought 3: There is no formal wife; Mary Austin is the closest answer.
Action 3: Finish[Freddie Mercury was never married; his closest partner was Mary Austin.]
```

Why ReAct vs CoT (Q10)

- CoT operates entirely in the model's head — no external knowledge.
- ReAct's **Observation** step injects fresh, grounded facts from tools. For knowledge-intensive tasks (current events, niche facts) ReAct is far more reliable.

17.3 Reflexion (Shinn et al., 2023)

Improves over **multiple trials** of the same task using *verbal self-reflection*:

```
Trial 1: agent attempts task, fails.
Self-reflection: "I failed because I assumed X. Next time I should check Y first."
Memory: append the reflection.
Trial 2: agent reads its memory before attempting, performs better.
```

No weight updates. Improvement comes entirely from natural-language memory accumulated across trials. This is the “verbal reinforcement learning” answer to Q1.19.

17.4 Self-Refine (Madaan et al., 2023)

A single-trial loop:

```
Initial output → Feedback → Refine → (loop until satisfied or budget exhausted)
```

- **Feedback stage** (Q9): the model (or an external critic) analyses the initial output and lists specific shortcomings (“This code is $O(n^2)$; consider a hash map,” “This paragraph repeats itself,” “The reasoning skipped step 3”).
- **Refine stage**: model rewrites the output using the feedback.

Self-Refine differs from Reflexion: Self-Refine iterates *within one trial*; Reflexion iterates *across trials* on the same task.

17.5 ReWOO — Reasoning Without Observation (Xu et al., 2023, Q1.17)

Idea. Decouple planning (which is expensive — many LLM calls in ReAct) from execution.

```
1. Planner LLM call: produce a *complete plan* using placeholders for tool outputs:
    Plan:
        #E1 = Search["singer of Bohemian Rhapsody"]
        #E2 = Search["wife of #E1"]
        Answer = Format("Wife of {E1} is {E2}")
2. Worker: execute each tool sequentially, filling in #E1, #E2, ...
3. Solver LLM call: take filled plan + evidence, produce final answer.
```

- Only **2 LLM calls** vs ReAct's many.
- Saves tokens and time.
- Limitation: if the plan is wrong, no chance to adapt mid-execution.

17.6 HuggingGPT (Shen et al., 2023)

LLM as controller of a *pipeline* of specialist models. Four stages: 1. **Task Planning** (Q1.18). Parse user request into a structured DAG of sub-tasks with input/output types and dependencies. 2. **Model Selection**. For each sub-task, pick a Hugging Face model best suited to it. 3. **Task Execution**. Run each model, pass outputs to dependent tasks. 4. **Response Generation**. Aggregate results into a final natural-language response.

The Task Planning stage is the one that **parses the user request into structured sub-tasks** — answer **B** in Q1.18.

17.7 Comparison

	Tools?	Loop?	Memory?	Trials?	When to use
CoT	no	no	no	1	reasoning, no external knowledge
ReAct	yes	per-step	no	1	knowledge-intensive QA
ReWOO	yes	plan-then-exec	no	1	when token cost matters
Self-Refine	no	refine	per-trial	1	output quality refinement
Reflexion	yes (optional)	across trials	persistent	many	tasks with stochastic feedback
HuggingGPT	yes (HF models)	once per plan	no	1	multi-modal pipelines

18. Worked numerical problems

18.1 Perplexity

A unigram LM gives $P(\text{"the"}) = 0.10$, $P(\text{"cat"}) = 0.01$, $P(\text{"sat"}) = 0.05$. Compute the perplexity of the test sentence "the cat sat".

```
log P = log 0.10 + log 0.01 + log 0.05 = -2.303 - 4.605 - 2.996 = -9.904
H = -(-9.904) / 3 = 3.301 nats
PPL = exp(3.301) ≈ 27.1.
```

18.2 Add-1 smoothing

Vocab size $V = 1000$. Observed $c(w_{t-1}, w_t) = 0$, $c(w_{t-1}) = 50$. - MLE: $0 / 50 = 0$. - Add-1: $(0 + 1) / (50 + 1000) = 1 / 1050 \approx 9.52 \times 10^{-4}$.

18.3 Self-attention dimensions

Given $T = 4$, $d = 8$, single head: - $Q, K, V \in \mathbb{R}^{4 \times 8}$. - $A = \text{softmax}(Q K^T / \sqrt{8}) \in \mathbb{R}^{4 \times 4}$. - Output $A \cdot V \in \mathbb{R}^{4 \times 8}$.

18.4 Multi-head split

$d_{\text{model}} = 512$, $h = 8 \rightarrow d_k = d_v = 64$. Each head has its own $512 \rightarrow 64$ projections; 8 outputs of 64 are concatenated back to 512 and projected by $W_0 \in \mathbb{R}^{512 \times 512}$.

18.5 LSTM parameter count

For $m = 256$ input, $n = 512$ hidden: - Each gate: weight $(m + n) \cdot n = 768 \cdot 512 = 393,216$ + bias $n = 512 \rightarrow 393,728$. - Four gates $\rightarrow 4 \cdot 393,728 \approx 1.57$ M params per LSTM layer.

18.6 Bradley–Terry probability

RM gives $r(x, y_w) = 2.5$, $r(x, y_1) = 1.0$. Probability of preferring y_w :

$$P(y_w > y_1) = \sigma(2.5 - 1.0) = \sigma(1.5) = 1 / (1 + e^{-1.5}) \approx 0.818.$$

18.7 PPO clip case analysis

Suppose $\hat{A}_t = +2.0$, $\epsilon = 0.2$, $\pi_\theta(a_t|s_t) = 0.9$, $\pi_{old}(a_t|s_t) = 0.5$. Then $r_t = 1.8 > 1.2$.
- Unclipped: $r_t \cdot \hat{A}_t = 3.6$. - Clipped: $1.2 \cdot \hat{A}_t = 2.4$. - $\min(3.6, 2.4) = 2.4 \rightarrow$ surrogate is the clipped version $\rightarrow$ gradient is zero w.r.t. θ at this step (it's a constant). Update stops.

18.8 Transformer parameter count (one block)

For $d = 768$, FFN ratio 4: - MHA params: 4 projections of size $d \times d = 768^2 = 589,824$ each $\rightarrow 4 \cdot 589,824 \approx 2.36$ M. - FFN params: $W_1 \in \mathbb{R}^{d \times 4d} = 768 \cdot 3072 \approx 2.36$ M, $W_2 \in \mathbb{R}^{4d \times d}$ similar $\rightarrow \approx 4.72$ M. - LayerNorm (2 LNs): $2 \cdot 2 \cdot d \approx 3$ K (negligible). - **Total ≈ 7.08 M per block.** For BERT-base (12 blocks): $12 \times 7.08 + \text{emb} \approx 110$ M. ✓

18.9 Mask matrix for $T = 4$

```
[[ 0, -∞, -∞, -∞],
 [ 0,  0, -∞, -∞],
 [ 0,  0,  0, -∞],
 [ 0,  0,  0,  0]]
```

Add this to scaled scores before softmax.

18.10 KL effect with extreme β

With $\beta \rightarrow \infty$, the KL term dominates $\rightarrow \pi_\theta \approx \pi_{ref} \rightarrow$ no reward optimisation. With $\beta \rightarrow 0$, no regularisation $\rightarrow$ policy may drift arbitrarily and reward-hack. Typical β in InstructGPT-scale runs: 0.01 to 0.1.

18.11 GloVe vs word2vec for OOV

Both produce static embeddings. For a word not in training vocabulary, neither has a vector. Modern subword embeddings (BPE, WordPiece in BERT; SentencePiece in T5) decompose unknown words into known subwords, side-stepping the OOV issue.

18.12 Beam search behaviour (Q1.5)

Claim: "as beam size increases, output probability decreases." False — opposite. A larger beam considers more hypotheses; the returned best is at least as good as that of a smaller beam (the smaller beam's choice is in the larger beam too). So **larger beam → higher (or equal) sequence probability**.

19. Appendix A — Confusion pairs

- **BERT vs GPT.** Bidirectional MLM (no native generation) vs causal LM (generation native).
- **T5 vs BART.** Both encoder–decoder. T5: span corruption + sentinels, text-to-text. BART: multi-noise denoising, especially strong at summarisation.
- **Prompt tuning vs Prefix tuning.** Both learn continuous prompts on a frozen LM. Prompt tuning: input layer only. Prefix tuning: K, V injected at every layer.
- **Bahdanau vs Luong.** Additive ($v^T \tanh(\dots)$), uses previous decoder state, vs multiplicative/dot, uses current decoder state.
- **RAG-Token vs RAG-Sequence.** Token: per-token marginalisation over passages. Sequence: per-output marginalisation; output is from one doc.
- **REINFORCE vs PPO.** Monte-Carlo on-policy vs clipped IS-weighted with multi-epoch reuse.
- **REALM vs FiD.** REALM marginalises over a single retrieved doc; FiD encodes K docs separately and fuses in the decoder.
- **ReAct vs ReWOO.** ReAct alternates Thought/Action/Observation each step (many LLM calls); ReWOO plans the whole chain first (one LLM call), then executes (worker), then solves (one LLM call) — far fewer LLM calls.
- **Self-Refine vs Reflexion.** Self-Refine: within-trial Feedback→Refine loop. Reflexion: across-trial verbal memory.

19. Appendix B — Model paper answer key with one-line reasons

Q	Answer	Reason
1.1	False	GloVe has no negative sampling.
1.2	B	Decoder-only can infer position from the causal mask itself.
1.3	A, D	REALM and RAG-Sequence give single-doc outputs.
1.4	True	GPT can implicitly count via attention.
1.5	False	Larger beam $\rightarrow$ higher (or equal) probability output.
1.6	False	RAG <i>reduces</i> hallucination relative to closed-book.
1.7	False	Multi-hop QA needs iterative retrieval.
1.8	False	Bahdanau attention is many-to-many.
1.9	False	Per-token inference latency of Transformer is $O(T \cdot d)$, not lower than RNN.
1.10	C	Sparse / dense indexes return top-K in $o(N)$.
1.11	B	Positional encoding adds order to bag-of-words self-attention.
1.12	C	Output = weighted sum of <i>values</i> .
1.13	B	Pre-training data isn't instruction-formatted.
1.14	C	Evol-Instruct uses an Instruction Evolver.
1.15	B	Environment = abstract text-input + output + reward.
1.16	B	Maximise $\log \sigma(r_w - r_l)$.
1.17	C	ReWOO decouples planning from tool calls.
1.18	B	Task Planning parses request into sub-tasks.
1.19	B	Reflexion = verbal RL via self-reflection.

Q	Answer	Reason
1.20	D	Decoder-only or encoder-decoder suffices for translation.

— end of detailed notes —